HRMS Microservices Architecture and Developer Implementation Guide

Developer handoff • Version 1.0 • 4 November 2026

Implement the HRMS using the microservice boundaries and engineering controls in this document. The target is an internal company system handling employee records, attendance, leave, documents and payroll. Microservices are an explicit requirement.

MUST indicates a release requirement. SHOULD indicates a preferred approach; record a reason and obtain technical review before deviating. This is a target specification based on the preceding source review, not confirmation that the existing implementation meets these requirements. The archive has not been re-executed for this document.

1 Implementation decisions

Decision	Instruction
Initial deployment	One company. Use fixed service database connections. Keep future tenant provisioning outside the request path.
Backend and frontend	Retain Java 21 and Spring Boot services and the Next.js BFF. Select supported dependency releases through a compatibility and vulnerability check.
Identity	Use OIDC with the company identity provider. Remove production account-picker login.
Data	Each service owns its database, migrations and runtime role. A shared managed PostgreSQL cluster is acceptable initially.
Messaging	Use RabbitMQ or a managed durable broker with persistent storage, publisher confirmation and consumer acknowledgements.
Release	A technical lead approves architecture deviations and evidence for the acceptance criteria. Business owners approve payroll policy.

2 Developer deliverables

- Submit source, service contracts, migrations, automated tests and CI configuration.
- Submit deployment manifests, example configuration without secrets, operational runbooks and a restore report.
- Record decisions for identity provider, broker, hosting, tenancy, payroll policy and recovery targets. Do not silently choose business policies.

3 Target service architecture

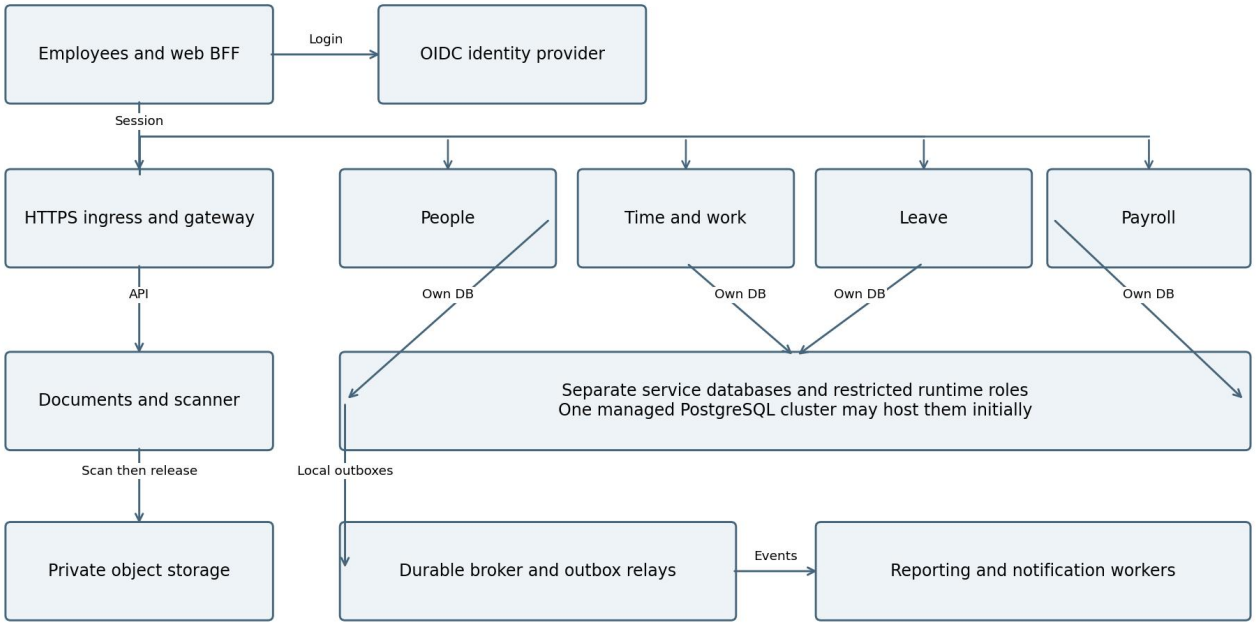


Figure 1 Service and data ownership. The gateway routes independently to each service; services do not share database access. Each service writes its own outbox in its own local transaction; relays publish events to the broker.

Service	Authoritative responsibility
People	Employees, organization, reporting history, employment periods and identity mapping
Time and Work	Schedules, attendance, corrections, tasks and work logs
Leave	Policies, approval chains, reservations, balances and immutable balance ledger
Payroll	Effective salary history, payroll runs, snapshots, revisions and finalization
Documents	Document links, classification, ACLs, scanning state and access audit
Reporting	Event-driven read models, export jobs and projection checkpoints
Notifications	In-app messages, delivery attempts, deduplication and retries

MUST NOT read or write another service database. Share versioned IDs and contracts. Keep tasks and work logs within Time and Work initially. Extract a Work Management Service only after its domain and deployment needs justify the change.

4 Identity and authorization

- MUST authenticate through OIDC authorization code flow with PKCE and verified state and nonce. Map the verified provider subject to an active employee record. Never accept employeeId as proof of login.
- MUST use secure HTTP-only cookies, CSRF controls and server-side session storage. Keep refresh tokens out of browser JavaScript. Rotate sessions after login and privilege changes.
- MUST validate issuer, audience, signature, allowed algorithms, expiry and signing-key rotation. The gateway performs edge validation; services validate service-audience credentials and enforce domain permissions.
- MUST strip client identity headers. Propagate user identity only in a verifiable token or signed assertion. Use workload-specific credentials for service calls, plus mTLS or equivalent private encrypted transport.
- MUST enforce feature permission and record scope together. An employee may view self, a manager only authorized reports, and HR only explicitly allowed records. Do not rely on hidden UI controls.
- MUST implement administrative session revocation and offboarding. Privileged payroll writes must check current authority; document any bounded cache delay for ordinary reads. Require MFA for privileged identities.

5 Data and transaction rules

- MUST use restricted runtime roles. Only a separate migration identity may change schema. Provisioning credentials must not be distributed to application services.
- MUST add uniqueness on canonical payroll employee and period keys, leave operation keys and other natural keys. Use typed status columns, foreign keys within a service and check constraints.
- MUST protect competing balance and state updates using optimistic version checks, explicit row locks or atomic conditional SQL. Define the retry behavior for conflicts.
- MUST store money as decimal with documented currency precision and rounding. Store instants in UTC and preserve location time zones for schedules and period boundaries.
- MUST retain finalized payroll revisions and rule/fact snapshots. Use explicit corrections and reversals rather than overwriting historical finals.
- MUST apply expand-and-contract migrations. Test upgrades from the previous production schema; application rollback must remain compatible or use a reviewed forward-recovery procedure.

6 Payroll finalization and recovery

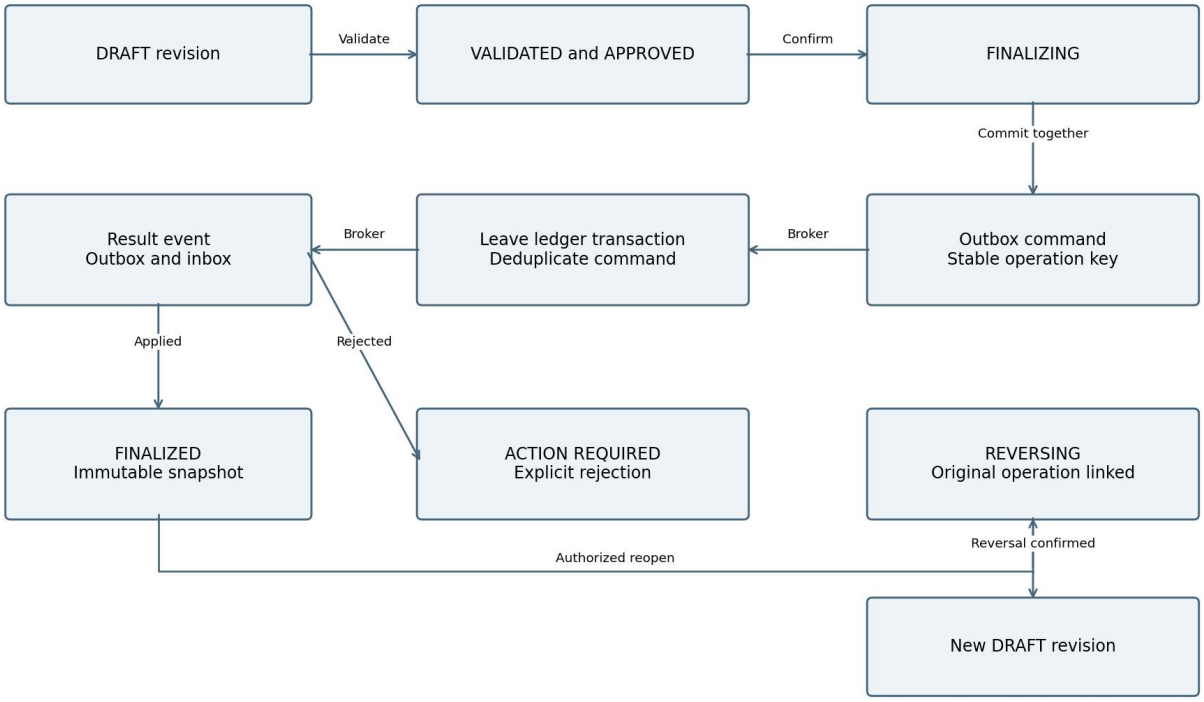


Figure 2 Finalization is an asynchronous recoverable operation. The result handler checks operation ID and revision before changing payroll state. A timeout means pending or unknown outcome, not permission to create a new deduction.

- Create a unique operation key from payroll run, payslip revision and action. Store FINALIZING and the outbox command atomically. Return HTTP 202 with operationId and status URL.
- Leave MUST record the command inbox key, ledger adjustments and result outbox in one local transaction. Duplicate commands return the recorded result. Reject invalid cover explicitly.
- Payroll MUST consume the result idempotently and transition the matching revision to FINALIZED or ACTION_REQUIRED. Ignore stale or already processed results safely.
- On reopen, retain the original final and initiate a separately keyed reversal. Create a new editable revision only after reversal confirmation.
- Run reconciliation for pending operations and ledger totals. Alert on age thresholds; provide safe replay and manual resolution with reason and audit.

Transport guarantee: at-least-once delivery with idempotent consumers. Do not claim exactly-once execution. Salary calculations must split effective-date segments when required by the approved policy and include leavers employed during the period.

7 Contracts and service resilience

Contract	Required behavior
Synchronous APIs	Publish OpenAPI and protobuf contracts. Version incompatible changes. Use typed DTOs, validation and a common error envelope with code and correlationId.
Lists and exports	Use one zero-based page convention with bounded size and deterministic sort. Filter and aggregate in SQL. Export complete date ranges through background jobs.
Deadlines	Set explicit HTTP and gRPC deadlines. Bound connection, request and worker budgets. No long database transaction may wait on a remote call.
Retries	Use exponential backoff and jitter with limits. Retry only safe or idempotently keyed operations. Route exhausted messages to a monitored dead-letter queue.
Events	Include eventId, eventType, schemaVersion, aggregateId, aggregateVersion, occurredAt, correlationId and payload. Include tenantId when tenancy is enabled.
Outbox and inbox	Publish after local commit using a durable relay. Confirm broker publication before marking sent. Deduplicate consumers; acknowledge after durable processing.
Ordering	Partition or route by aggregate where order matters. Detect stale versions and gaps. Provide replay and projection rebuild procedures.

8 Document security

- MUST store document metadata and employee links in Document Service. Validate file existence and attachment authority before creating the link.
- MUST quarantine uploads; check size, file signature and document structure, then scan before release. MIME labels alone are insufficient. Keep storage private.
- MUST authorize each download against document classification and employee scope. Issue short-lived URLs only after authorization; audit issuance and actual access where supported.
- MUST define retention, deletion, scan-failure handling and orphan cleanup. A failed metadata commit must not leave untracked objects indefinitely.

9 Production deployment

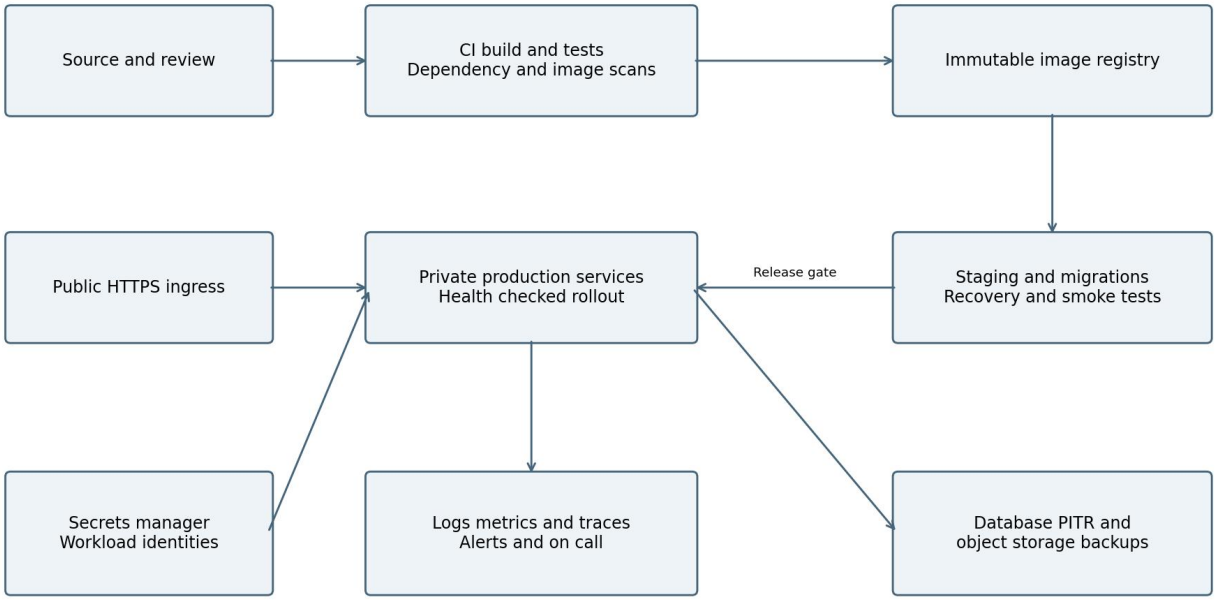


Figure 3 Production topology and release route. Use a maintained managed container platform; Kubernetes is optional. Only the HTTPS ingress is public. Migration jobs connect directly to the relevant databases using a separate identity.

- MUST build each artifact once in CI and promote the same image digest. Run containers as non-root with resource limits and least filesystem permissions.
- MUST expose liveness and readiness separately, handle graceful shutdown and test rolling deployments. Use sufficient replicas and managed infrastructure availability to meet agreed targets.
- MUST keep databases, broker, object storage and service ports private. Use secrets management, credential rotation and startup rejection of development credentials.
- MUST collect structured logs, metrics and traces. Propagate correlation IDs across HTTP, gRPC and messages; exclude tokens and sensitive document contents from logs.
- MUST monitor latency, errors, saturation, DB pool usage, queue lag, outbox backlog, dead-letter volume, scan failures and payroll reconciliation discrepancies.

10 Recovery and operational procedures

Before deployment, agree the service availability objective, recovery time objective and recovery point objective. Proposed starting targets for business review are 99.5 percent monthly availability, four-hour recovery and no more than fifteen minutes of committed database data loss. These are proposals, not established guarantees; validate object-storage recovery separately and state cross-service consistency limits.

- Enable encrypted database backups and point-in-time recovery. Back up approved documents, configuration and required broker state. Define retention and independent access controls.
- Demonstrate restoration into an isolated environment. Recover to a documented point, verify document references and reconcile payroll against leave ledgers and operation IDs.
- Define owners and escalation routes for identity outage, broker outage, database outage, stuck payroll, failed migration, malicious upload and credential exposure.
- Document replay, rollback or forward recovery, emergency access and key rotation. Every manual financial correction requires reason, actor and before/after references.

11 Existing implementation changes

Current area	Developer action
Account picker and token endpoint	Replace with verified identity flow; remove production impersonation path.
Payroll finalization and leave cover	Replace remote mutation inside local transaction with the durable workflow on page 4.
Payslip and leave balance persistence	Add database uniqueness, concurrency protection and ledger operation deduplication.
Monthly payroll and salary resolution	Select employment overlap and implement approved effective-date salary segmentation.
Dynamic tenant routing and pools	Use fixed connections for this deployment. If retained, fix capacity and lifecycle races and handle Redis failure explicitly.
Attendance reports and task lists	Query bounded date ranges and database aggregates. Remove single-page historical export assumptions.
Audit and document access	Derive actor server-side; align access policy and audit with authoritative operations.
Placeholder tests and local Compose	Replace with behavioral gates and an enforced production deployment profile.

12 Implementation sequence and review gates

Stage	Required output	Exit condition
1 Foundation	Service boundaries, identity mapping, API conventions, schema ownership and CI	Authentication and cross-role negative tests pass.
2 Data integrity	Constraints, state machines, operation IDs, snapshots and audit model	Concurrent requests preserve balances and unique payroll records.
3 Distributed workflows	Outbox/inbox, broker, finalization, reversals and reconciliation	Crash and duplicate-delivery tests recover correctly.
4 Features and reporting	Employment rules, salary segmentation, document ACLs and full exports	Approved examples match independently calculated results.
5 Operations	Private deployment, alerts, backups and runbooks	Restore and release-recovery drills succeed.
6 Pilot	Limited approved users and parallel payroll comparison	Business and technical owners accept the evidence.

13 Mandatory acceptance tests

- SEC01 Unauthenticated visitors cannot obtain a session by supplying an employee ID. Employees cannot read or mutate another employee record outside allowed scope.
- DATA01 Concurrent calculate calls create at most one current payslip per employee and period. Concurrent balance changes match the ledger sum.
- PAY01 Duplicate finalize commands and lost responses produce one leave deduction. A crash after HRMS commit recovers without another deduction.
- PAY02 Duplicate reversals restore leave once. Stale results cannot finalize a newer revision. A business rejection remains visible and recoverable.
- PAY03 Joining, leaving, mid-month salary change, unpaid leave and rounding match signed-off expected values.
- REP01 Exports for old months remain complete after several years of attendance; page limits do not truncate them.
- DOC01 Wrong-role downloads fail; unscanned files cannot be downloaded; authorized HR can access permitted employee documents.
- OPS01 Broker and database interruptions recover predictably. A backup restore validates documents, balances, payroll revisions and pending operations.

For each test, submit environment, dataset, steps, expected result, actual result and automated evidence. Source-string assertions and console success messages do not satisfy these tests.

14 Definition of done and submission checklist

- All critical security and financial integrity defects are closed with regression tests. No demo authentication or known default secrets can start in production.
- Each service has an owner, data model, migrations, API/event contracts and documented dependencies. No service accesses another service database.
- CI runs unit, PostgreSQL integration, contract, authorization and critical frontend tests; it scans dependencies and images and retains reports.
- Load tests use realistic users and historical volumes. Record p95 latency, errors, database connections and queue backlog against agreed budgets.
- A deployed staging environment demonstrates message retry, deduplication, reconciliation, offboarding, document access and safe deployment.
- A restore drill and release-recovery drill are complete, with timings, limitations and remediation items recorded.
- Payroll policy, recovery targets, retention and tenancy decisions have named business or technical approvers.
- Submit release notes, known limitations, runbooks and rollback or forward-recovery instructions. Technical lead reviews the evidence before production approval.

15 Decisions to record before development

Decision	Owner	Record required
Identity provider and MFA	Technical lead and company IT	Provider, employee mapping, emergency access and revocation policy
Payroll and leave policy	HR and Finance	Salary segmentation, leave cover, rounding and approval separation
Tenant scope	Product owner and technical lead	Single-company launch or committed multi-company requirements
Availability and recovery	Business owner and operations	SLO, RTO, RPO, backup retention and monitoring thresholds
Hosting and broker	Technical lead and operations	Managed platform, connectivity, capacity and ownership
Data retention	Business owner and company IT	Employee documents, payroll, audit and deletion schedule

Developer instruction: follow the specified boundaries and MUST requirements. Raise a written design decision before changing ownership, weakening an integrity guarantee or introducing another service. Keep implementation choices proportionate to the internal HRMS scope.